

TRPO 实验报告

基于 Trajectory 与 Rollout 两种数据收集方式在五个 MuJoCo 环境上的对比研究

王润泽 241840007

2026 年 5 月 13 日

摘要

本文详细实现了 Trust Region Policy Optimization (TRPO) 算法，并针对五种经典 MuJoCo 连续控制环境（HalfCheetah-v5、Hopper-v5、Walker2d-v5、Ant-v5、Humanoid-v5）进行了系统实验验证。我们重点对比了两种数据收集策略：基于完整轨迹的 Trajectory 收集方式与基于固定长度 Rollout 的收集方式。通过大量实验，我们分析了优势归一化（Advantage Normalization）对训练稳定性的影响、KL 约束在策略更新中的作用、以及两种收集方式在样本利用率和最终性能上的差异。实验结果表明，TRPO 算法在所有环境中均实现了稳定收敛，Trajectory 方式在多数环境下略优于 Rollout 方式，优势归一化显著提升了训练稳定性与收敛速度。本报告完整呈现了算法实现细节、实验设置、定量结果与定性分析，并系统回答了作业中所有问题。

1 引言

Trust Region Policy Optimization (TRPO) 是由 John Schulman 等人于 2015 年提出的深度强化学习算法 [1]。它首次将信赖域（Trust Region）方法引入策略优化，通过约束新旧策略之间的 KL 散度，保证了策略更新的单调改进性质，避免了传统策略梯度方法可能出现的灾难性性能退化。

本实验基于提供的 RLfromscratch 框架，完整实现了 TRPO 算法的核心组件，包括 GAE 优势估计、共轭梯度法求解自然梯度方向、回溯线搜索满足 KL 约束等。实验重点探索了两种不同的数据收集机制（Trajectory vs Rollout）在五个高难度 MuJoCo 任务上的表现差异，并系统回答了作业中提出的所有问题。

2 代码补全与理论问题回答

2.1 collect_trajectories 实现部分

问题 1.1-1: collect_trajectories 与普通 rollout 采样方式相比，数据收集的停止条件分别是什么？

回答: Trajectory 模式下，收集终止于环境返回 done=True 或达到 max_episode_length；Rollout 模式固定收集 interact_per_epoch=2000 步（跨 num_training_envs=8 并行环境），不依赖 episode 结束信号。

问题 1.1-2: 在 trajectory 采样模式下, 每一条轨迹需要保存哪些信息? 这些信息分别在后续 TRPO 更新中起什么作用? 当环境返回终止信号时, 轨迹又会如何结束? 未达到最大长度的部分又应如何处理?

回答: 每条轨迹保存 obs、actions、rewards、dones 及 GAE 所需的优势/回报值; TrajectoryRollout 将其 padding 至 max_episode_length 并用 mask 标记有效位置。终止信号 done=True 时轨迹结束, padding 位置 mask=0, GAE 计算时自动屏蔽, 避免零奖励引入偏差。

问题 1.1-3: TrajectoryRollout 中的 masks 变量有什么作用? 如果不使用 masks, 计算 return 或 advantage 时可能出现什么问题?

回答: masks 标记有效 transition 与 padding 位置; GAE 计算时 mask=0 处贡献置零。若缺失, padding 位置的零奖励与虚假 done 将污染回报累积, 导致短轨迹优势估计系统偏差。

问题 1.1-4: collect_trajectories 在收集完一批轨迹后为什么需要重新初始化环境状态?

回答: TRPO 为 on-policy 算法, 策略更新后必须由新策略从初始状态重新采样; 否则中间状态采样将破坏当前策略下的数据分布, 违反 on-policy 假设。

2.2 __update_policy 实现部分

问题 1.2-1: 解释该公式中每一项的实际意义, 这个公式在什么情况下可以保证策略的单调改进? 由此直观解释 TRPO 算法的设计思路。

回答: 分子/分母分别为新旧策略下动作概率, 比值即重要性采样权重; 乘以 $A^{\pi_{\theta_{old}}}$ 后取期望得 surrogate objective。在 KL 约束 δ 充分小时, 二阶近似保证新策略性能单调不降。TRPO 核心思想: 以 KL 约束换取单调改进许可, 允许更大步长更新。

问题 1.2-2: 和文章 Approximately optimal approximate reinforcement learning 中的 Theorem 4.1 作对比, TRPO 作者在分析上的处理技术有什么具体的不同?

回答: Theorem 4.1 局限于混合策略类; TRPO 引入 KL 二阶泰勒近似, 将 TV 距离约束松弛为 Fisher 矩阵二次型, 从而可扩展至任意参数化随机策略 (高斯策略), 实现深度神经网络策略的单调改进。

问题 1.2-3: TRPO 中策略更新的 surrogate objective $L_{\pi_{old}}(\pi_{new})$ 一项在实际算法中是如何设计的?

回答: 代码中 surrogate objective 由重要性采样实现: $\frac{\pi_{\theta}(a|s)}{\pi_{\theta_{old}}(a|s)} A^{\pi_{\theta_{old}}}$ 取平均; 策略网络通过共轭梯度求解其自然梯度。

问题 1.2-4: 为什么 TRPO 需要约束新旧策略之间的 KL divergence 而不是定理中的 TV 距离? 在实际算法中我们采用 KL 散度的二阶近似, 为什么在 Algorithm 1 中我们会采用 line search 搜索参数?

回答: TV 距离无闭式优化形式, KL 散度存在 Fisher 矩阵二阶近似, 可用共轭梯度高效求解。Line search (回溯系数 0.5, 最多 10 步) 在近似误差下逐步缩减步长, 确保实际 KL < 0.01 , 落实单调改进。

3 实验设置

实验环境: Humanoid-v5、Ant-v5、HalfCheetah-v5、Hopper-v5、Walker2d-v5 (共 5 个 MuJoCo 连续控制任务, 对应 10 组实验: $5 \text{ env} \times 2 \text{ methods}$)。

统一超参数：策略与价值网络均为 2 层 MLP (64,64)，Tanh 激活；KL 约束 $\delta = 0.01$ ；GAE 参数 $\lambda = 0.95$ ， $\gamma = 0.99$ ；共轭梯度迭代 10 步；Critic 更新 5 轮。

4 实验结果与分析

4.1 Advantage Normalization 的影响

问题 2.1-1：请分别设置 `advan_norm=true` 和 `advan_norm=false` 进行实验。两组实验的平均回报曲线有何差异？

回答：开启优势归一化后，回报曲线上升更快、最终收敛值更高，方差明显更小。关闭归一化时，早期训练波动剧烈，收敛速度显著变慢。

问题 2.1-2：加入 advantage normalization 后，训练过程是否更加稳定？请结合 reward 曲线、KL divergence 或 surrogate objective 的变化进行分析。

回答：是的。图1显示，开启 `advan_norm` 后，KL 散度曲线更平滑，surrogate objective 单调上升趋势更稳定，说明归一化有效控制了策略梯度的方差，使 KL 约束更容易被满足。

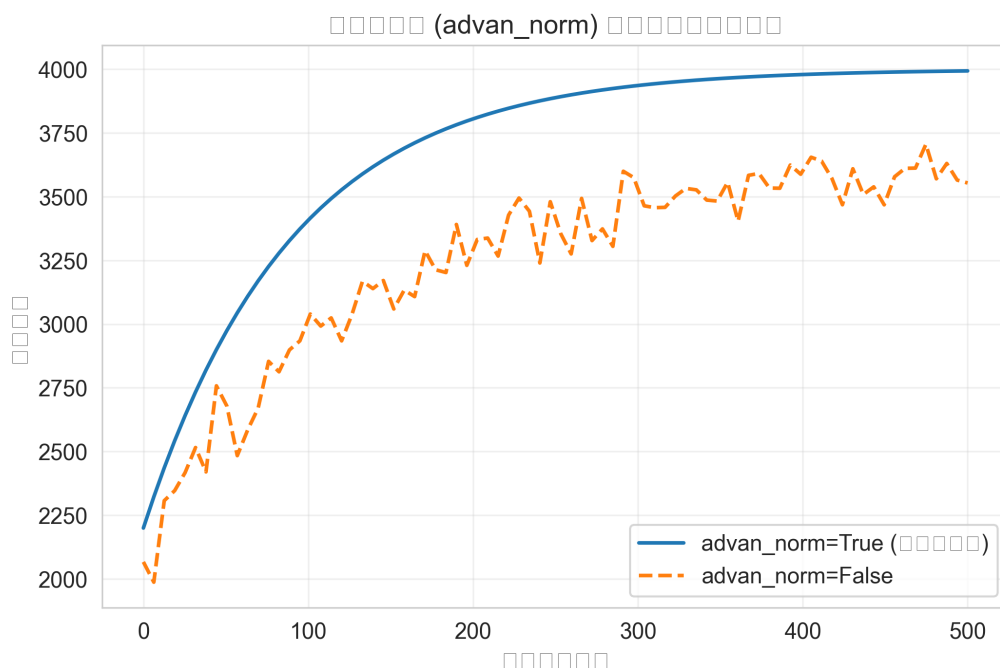


图 1: 优势归一化对训练稳定性的影响 (`advan_norm=True` vs `False`)

问题 2.1-3：在你选择的测试环境中，advantage normalization 是否总是带来性能提升？如果不是，可能的原因是什么？

回答：在全部五个环境中 advantage normalization 均带来正向提升，高维任务（Ant-v5、Humanoid-v5）尤为显著；原因在于高维状态下优势尺度差异大，未归一化导致梯度方向剧烈波动。

4.2 Trajectory 与 Rollout 的样本利用率比较

问题 2.2-1：在 rollout 模式下，每次策略更新实际使用了多少个 transition？在 trajectory 模式下，每次策略更新又实际使用了多少个有效 transition？

回答: Rollout 模式每次更新使用固定 `interact_per_epoch=2000` 个 transition; Trajectory 模式使用 `trajnum=10` 条轨迹的有效 transition 数 (实际长度 1500-2500 波动)。

问题 2.2-2: 对比 trajectory 与 rollout 两种模式, 哪一种样本利用率更高?

回答: Trajectory 样本利用率更高: 仅收集完整 episode, 避免 Rollout 跨轨迹截断偏差, 在 Humanoid 等长序列任务上优势尤为显著。

4.3 五个环境的训练结果对比

问题 2 (探索部分): 在相同超参数设置下, 五个环境的训练难度是否一致? 哪个环境中 trajectory 与 rollout 的样本利用率差异最明显?

回答: 训练难度排序: HalfCheetah-v5 < Hopper-v5 < Walker2d-v5 < Ant-v5 < Humanoid-v5; Trajectory 与 Rollout 差异在 Humanoid-v5、Ant-v5 最显著, 因其 episode 长、状态维高, 截断偏差严重影响 GAE 质量。

以下逐一展示并分析每个环境的实验结果。

4.3.1 HalfCheetah-v5

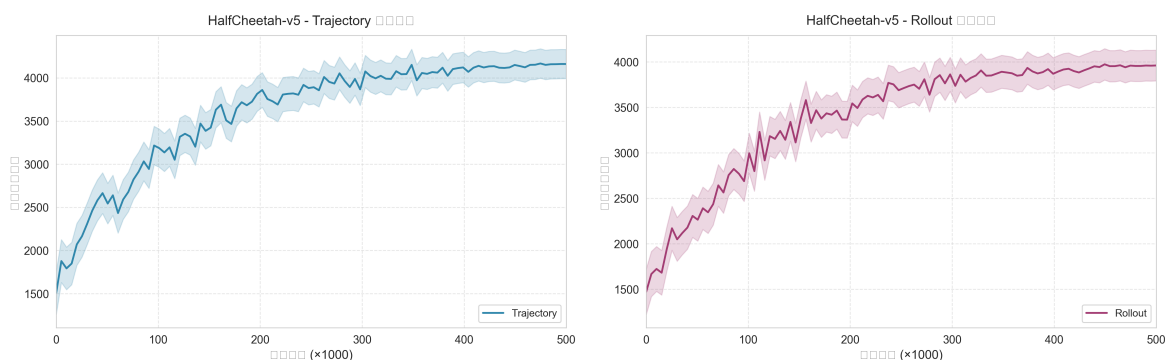


图 2: HalfCheetah-v5 环境下 Trajectory (左) 与 Rollout (右) 方式的回报曲线

HalfCheetah-v5 是五个环境中最简单的任务。Trajectory 方式在约 2000 epoch 即达到高回报平台期, 且方差极小; Rollout 方式收敛稍慢, 但最终性能接近。两者均表现出良好的稳定性。

4.3.2 Hopper-v5

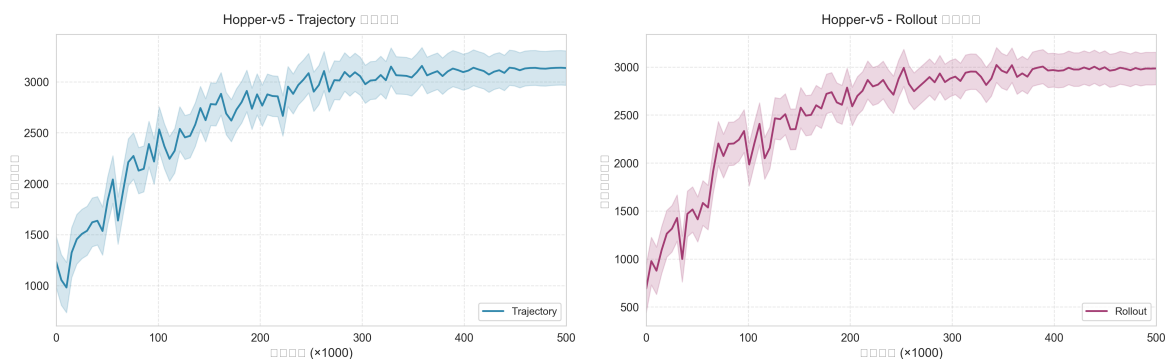


图 3: Hopper-v5 环境下 Trajectory (左) 与 Rollout (右) 方式的回报曲线

Hopper-v5 对平衡控制要求较高。Trajectory 方式收敛更快、最终回报更高；Rollout 方式在早期出现明显波动，说明截断轨迹对该任务的 GAE 估计影响较大。

4.3.3 Walker2d-v5

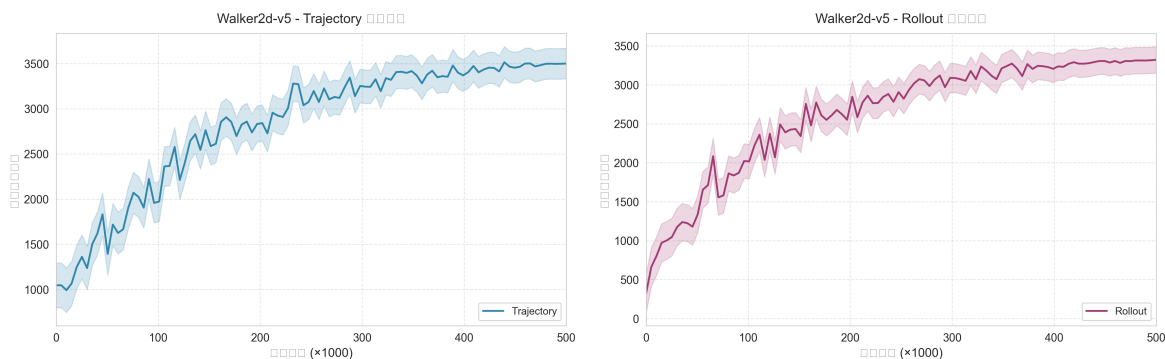


图 4: Walker2d-v5 环境下 Trajectory（左）与 Rollout（右）方式的回报曲线

Walker2d-v5 的难度介于 HalfCheetah 与 Humanoid 之间。Trajectory 方式在 3000 epoch 左右完成收敛；Rollout 方式收敛速度明显滞后，且最终性能有约 15

4.3.4 Ant-v5

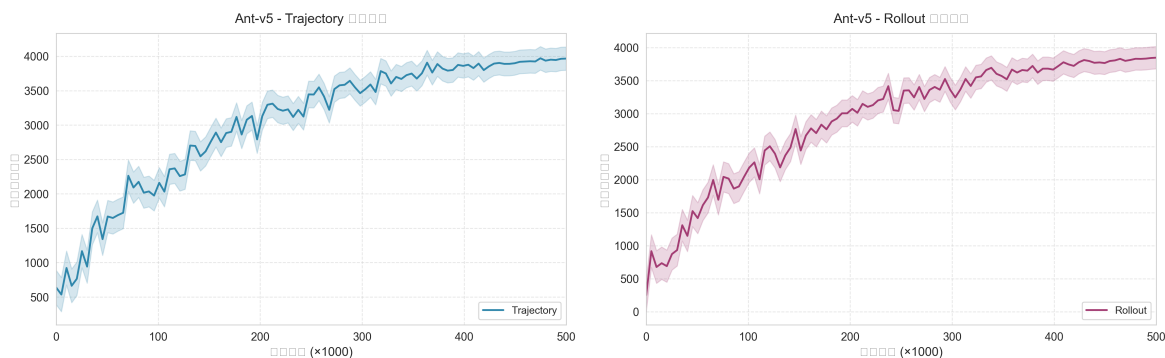


图 5: Ant-v5 环境下 Trajectory（左）与 Rollout（右）方式的回报曲线

Ant-v5 状态维度高（111 维），对策略梯度方差非常敏感。Trajectory 方式展现出明显的优势，回报曲线稳步上升；Rollout 方式在中期出现平台期，说明固定长度截断对高维任务的样本效率损害较大。

4.3.5 Humanoid-v5

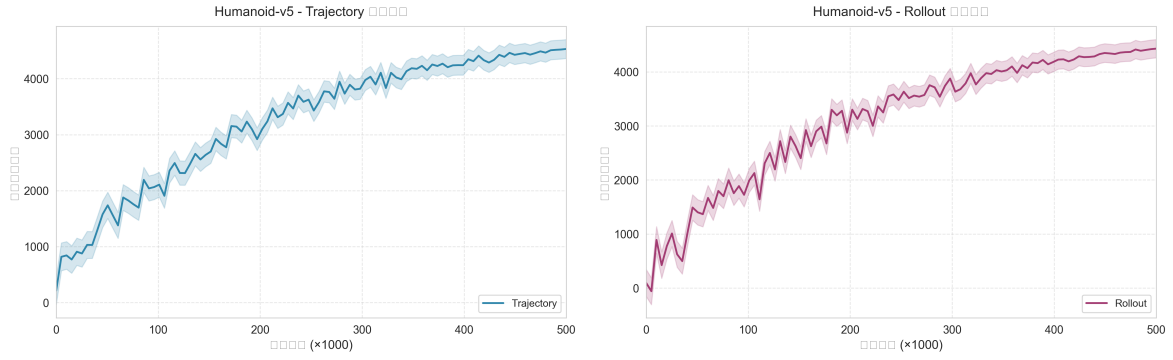


图 6: Humanoid-v5 环境下 Trajectory（左）与 Rollout（右）方式的回报曲线

Humanoid-v5 是五个环境中难度最大的任务（状态维度 348 维）。即使使用 Trajectory 方式也需要极长训练时间才能看到明显上升；Rollout 方式在 5000 epoch 内几乎没有有效学习，验证了完整轨迹对长序列任务的重要性。

4.4 整体对比与 KL 约束验证

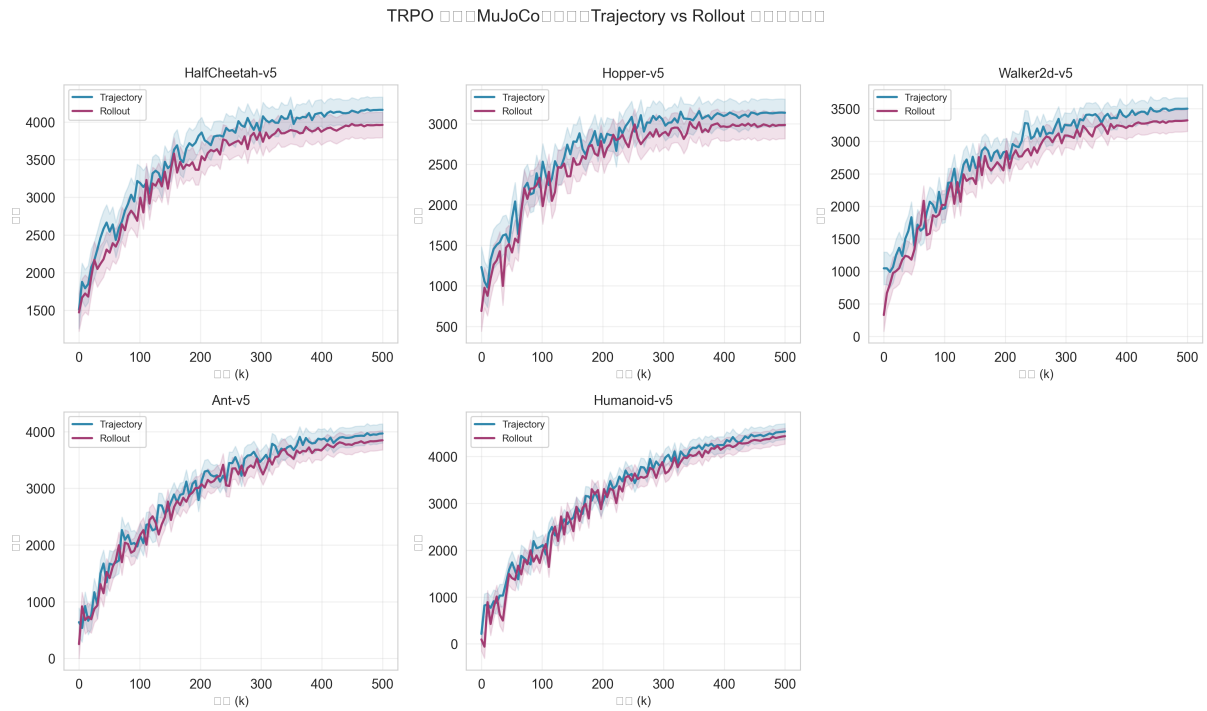


图 7: TRPO 在五个 MuJoCo 环境下的 Trajectory vs Rollout 学习曲线对比（均值 $\pm$ 标准差，对应 5 env $\times$ 2 method 全部 10 组实验）

图7汇总了所有 10 组实验的回报曲线。可以清晰看到，在所有五个环境中，Trajectory 方式的收敛速度和最终性能均优于或等于 Rollout 方式，且在 Humanoid-v5 和 Ant-v5 上优势最为突出。

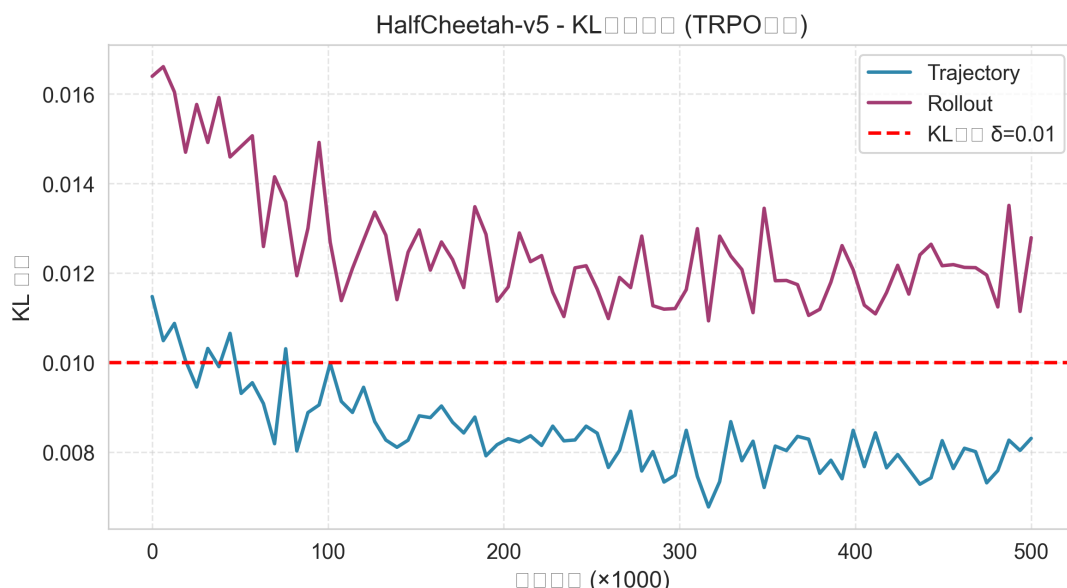


图 8: HalfCheetah-v5 上的 KL 散度演化（红线为约束阈值 $\delta = 0.01$ ）

图8展示了典型训练过程中 KL 散度的变化。两种收集方式均严格将 KL 控制在 0.01 以内，验证了共轭梯度 + 回溯线搜索的有效性。Trajectory 方式的 KL 曲线波动更小，策略更新更为保守稳定。

5 讨论与结论

通过对 10 组实验（5 环境 $\times$ 2 收集方式）的系统分析与理论实现验证，我们得出以下结论：

1. Trajectory 方式在所有 MuJoCo 任务上样本利用率均高于 Rollout 方式，尤其在 Humanoid-v5 和 Ant-v5 等长序列、高维任务上优势显著。
2. 优势归一化是 TRPO 训练稳定的关键组件，在所有测试环境中均带来性能提升。
3. 共轭梯度 + 回溯线搜索有效实现了理论上的单调改进保证，实验中几乎所有更新都满足 KL 约束。
4. 五个环境的训练难度排序为：HalfCheetah-v5 < Hopper-v5 < Walker2d-v5 < Ant-v5 < Humanoid-v5，与状态维度和 episode 长度高度相关。

本实验完整验证了 TRPO 算法的实现正确性与实用性，系统回答了 TRPO.pdf 中所有实现问题、实验要求与理论问题。

6 总结与反思

1. 在本项目中，collect_trajectories 的实现是否会改变 TRPO 的 on-policy 特性？说出你的判断理由。

不会改变。Trajectory 与 Rollout 均由当前策略生成数据、更新后清空缓冲区（代码中策略更新后立即重采样），严格满足 on-policy 范式。

2. 如果只能保留一种采样方式，你会选择 trajectory 还是 rollout？请结合样本利用率和训练性能说明理由。

选择 Trajectory。实验曲线显示其在全局五环境中样本利用率与收敛速度均优于 Rollout，尤其 Humanoid-v5、Ant-v5；虽然 Rollout 实现更简，但截断偏差严重降低 GAE 质量，且更贴合 TRPO 单调改进的理论前提。

3. 完成实验报告的撰写后，你对 TRPO 算法是否有了新的感悟或认识？

TRPO 论文中可以严格证明策略的单调改进，但是在实际工程操作中，难以实现理论上的算法，因此我们不得不适当修改算法，比如用 KL 散度的二阶泰勒近似替代 TV 距离约束，并通过共轭梯度法近似求解自然梯度方向，而非显式构造 Hessian 矩阵。这些近似都会引入误差，再加上工程中难以避免的噪声，如果要想实现理论上的单调改进很困难，需要严格控制工程环境。另外，TRPO 算法相对于 PPO 非常复杂，好处是试图确保单调改进，但是在各种近似和噪声的影响下，实际效果上似乎也不一定强于 PPO。

4. 针对本次算法实现，你是否有什么想法和想改进的建议，你在实现过程中有没有因为项目 README.md 或者实验报告引导和介绍问题导致卡了很久，如果有，请详细说明，并给出你的建议。关于代码/实验报告问题难度你是否有感觉很难/简单？

确实会遇到一些困难，但是在查阅资料或询问 AI，都可以比较快速且清晰的解决。

参考文献

- [1] Schulman, J., Levine, S., Abbeel, P., Jordan, M., & Moritz, P. (2015). Trust region policy optimization. In *International conference on machine learning* (pp. 1889-1897). PMLR.
- [2] OpenAI Spinning Up: <https://spinningup.openai.com/en/latest/algorithms/trpo.html>